

Defining References (`refs.dtx`)

Charles Duan

September 6, 2026

A reference represents a single citable work of authority. What constitutes a single work may be uncertain, particularly for statutes and multi-authored works, as described previously. The general test for whether two items are two references or subdivisions of a single reference is whether, if they were cited immediately in sequence, *id.* should be used for the second item. In some cases this is a matter of the writer's taste, so this package generally gives writers the flexibility to define references however they see fit.

If a reference is used in a citation string but not defined, this package will issue an error, or it will keep track of the missing citation. See `draw.dtx` for more information on missing references.

1 Data Model

Each reference consists of:

- A reference type, such as a book, case, or statute. The complete list of reference types is given in Part ???. The reference type determines how the reference is formatted and what parameters are necessary to define the reference.
- An identifier nickname that will be used to identify this reference in citations (and, on occasion, inside other references). The nickname must be unique and generally should not contain spaces.
- Parameters providing information about the reference. For example, a book uses parameters for the author, title, and publication year. The reference type determines which parameters are required and optional; this is documented in Part ??. There are also parameters of general applicability to all reference types, described in `genparams.dtx`. The complete list of parameters is given in Part ??.
- A Table of Authorities category, described below.

2 Input Syntax

The package provides its own syntax for writers to use when defining references. As an alternative, `hibib.dtx` describes a compatibility layer that accepts BibLaTeX reference definitions, so long as the parameter names correspond with those of this package.

Generally, references are defined as follows:

```
\def<reference type> {<name>} {
  <param1> = <value1>,
  <param2> = <value2>,
  ...
}
```

For example, to define the historic case on copyrightability:

```
\defcase{baker}{
  parties=Baker v. Selden,
  cite=101 U.S. 99,
  year=1880,
}
```

The input follows TeX syntax and uses the `keyval` package, so if there are equal signs or commas in the parameter values, then the parameter should be surrounded with braces.

Entering the parameter names for every reference can be tedious, so for common reference types there are shortcuts described in `cparse.dtx`.

Creates a new reference definer macro. #1 is the type name (used to create a `\def<type>` macro), #2 a textual description (used only for generating documentation), #3 the default TOA category (see `toa.dtx`), and #4 the macro code to be run to define the reference. Executing the reference definition command should cause the definition of the following formatting macros of the form `\<prefix>@<ref-name>`, with prefixes given in `draw.dtx`.

For convenience of parameter definition macros, `\@this@case` is set to the name of the reference being defined.

```
\def\hi@newcite#1#2#3#4{%
  \HiDefaultTOACat{#1}{#3}%
  \global\@namedef{def#1}##1##2{%
    \ifundefined{fc@##1}{%
      \PackageError{hi@pkgnam}{%
        Citation for ##1 already defined
      }{%
        The citation for ##1 has already been defined.\MessageBreak
        Please select another name.
      }%
    }%
    \begingroup
    \def\@this@case{##1}%
    \hi@toacat@setdefault{##1}{#1}%
    \hi@param@set{#1}{##2}%
    #4%
    % \hi@bib@output
  \endgroup
}%
}
```

`\hi@alias@cite`

Aliases a cite definer macro to another.

```
\def\hi@alias@cite#1#2{\@namedef{def#1}{\@nameuse{def#2}}}
```

Within a reference definer macro, defines a new formatting macro of the form $\langle prefix \rangle @ \langle name \rangle$, where #1 is $\langle prefix \rangle$ and #2 is $\langle name \rangle$. Mainly, this checks to make sure that a full citation macro isn't being redefined (indicating that the writer has used a duplicate name).

If a non-full form is being defined and it already exists, then the existing definition is not overwritten.

```
\def\hi@newcite@form#1#2{%
  \ifundefined{#1@#2}{%
    \expandafter\protected@xdef\csname#1@#2\endcsname
  }{%
    \find@eq{fc}{#1}{%
      \PackageWarning\hi@pkgname{Refusing to redefine #1@#2}%
    }{%
      \hi@newcite@form@gobble
    }%
  }
}
\makeatfind@eq{fc}
\def\hi@newcite@form@gobble#1#1{\@gobble}
```

3 Implementation of Parameters

This implementation documentation now turns to how parameters are stored and set internally. For every defined parameter, there is a macro $\text{\KV@hi@}\langle param \rangle$ as created by the `keyval` package. There is also a macro $\text{\hi@kv@}\langle param \rangle$ defined to hold the value that the parameter is set to, which is used by most but not all parameters. By default, the $\text{\hi@kv@}\langle param \rangle$ variable is given a value that will generate an error if used in a citation, alerting the user to the parameter being necessary but missing.

This is a list of all the parameter registers. Each parameter is stored in the form $\text{\do}\langle param \rangle\{\{\langle description \rangle\}\}$.

```
\def\hi@params{}
```

Paranthetical parameters. We make a special token list of paranthetical creator commands, to be executed presumably upon the full citation.

```
\newtoks\hi@param@parens
```

Adds a paranthetical to the list. #1 is the paranthetical priority, #2 the text.

```
\def\hi@param@addparen#1#2{%
  \push@toks\hi@param@parens{%
    \hi@parens@add#1{#2}%
  }%
}
%
% Special token to set undefined parameters to.
%
% \begin{macrocode}
\DeclareRobustCommand\hi@param@undef[1]{%
  \PackageError\hi@pkgname{%
    Citation parameter \noexpand#1 missing in \@this@case
  }{Include the missing citation parameter.}%
  \textbf{???}%
}

\def\hi@undefine#1{\def#1{\hi@param@undef{\string#1}}}
```

Set all parameters to empty. This will be called after all the parameters have been created. Since reference definitions are executed inside groups, these initial settings should carry over into all definitions.

```
\def\hi@param@clear{%
  \def\do#1#2{\expandafter\hi@undefine\csname hi@kv@#1\endcsname}%
  \hi@params
  \hi@param@parens{}%
}
\AtEndOfPackage{\hi@param@clear}
```

Parameter definers. #1 is the parameter, #2 is an explanation, and anything that follows should be arguments to $\text{\define@key}$ excluding the keyset and parameter name.

```
\def\hi@defparam#1#2{%
  \g@addto@macro\hi@params{\do{#1}{#2}}%
  \define@key{hi}{#1}%
}
\def\hi@newparam#1#2{%
  \expand{\hi@defparam{#1}{#2}}%
  \expandafter\def\csname hi@kv@#1\endcsname{##1}%
}
```

```

    }i%
  }
  \def\hi@defparam@noval#1#2{%
    \hi@defparam{#1}{#2}[#1]%
  }
  \def\hi@newparam@noval#1#2{%
    \expand{\hi@defparam@noval{#1}{#2}}{%
      \expandafter\def\csname hi@kv@#1\endcsname{##1}%
    }i%
  }
  \def\hi@param@alias#1#2{%
    \g@addto@macro\hi@params{\do{#1}{Alias for #2}}%
    \edef\reserved@a{%
      \noexpand\def\expandafter\noexpand\csname KV@hi@#1\endcsname{%
        \expandafter\noexpand\csname KV@hi@#2\endcsname
      }%
    }\reserved@a
  }
}

```

\hi@param@set 3(a) Setting Parameters The following macro is used when executing a reference definer, to read and set all the parameters in preparation for constructing the citation macros. It looks for a parseable formatted citation (either by finding a semicolon in the parameters, or determining that the parameters contain no equal sign), and otherwise uses `keyval` to read the parameters. It also executes any post-parameter hooks. **#1** is the reference type name, and **#2** is the parameter set.

```

\def\hi@param@set#1#2{%
  \let\hi@param@set@hooks\empty
  \def\hi@kv@kind{#1}%
  \hi@param@read{#2}%
  \hi@param@set@hooks
}
\def\hi@param@read#1{%
  \findin{;}{#1}%
  \hi@param@read@{#1}%
  \hi@param@read@{#1}{#1}{}%
}
\def\hi@param@read@#1#2#3{%
  \findin{=}{#2}{%
    \@firstofthree{\setkeys{hi}{#1}}%
  }{%
    \chop@space@then@run{#2}\hi@cparse
    \ifstrempy{#3}{\setkeys{hi}{#3}}%
  }%
}
\makeatfindin{=}
\makeatfindin{;}

```

Adds a post-parameter-setting hook. **#1** is code to be run.

```

\def\hi@param@addhook#1{%
  \addto@macro\hi@param@set@hooks{#1}%
}

```

3(b) Testing Parameters in Citation Macros Commands for testing parameters in citation macros.

```

\def\hi@ifset#1{%
  \expandafter\hi@ifset@#1.\@stop
}
\def\hi@ifset@#1#2\@stop{%
  \ifx\hi@param@undef#1\expandafter\@secondoftwo
  \else \expandafter\@firstoftwo \fi
}

\def\hi@ifeitherset#1#2{%
  \hi@ifset#1{\@firstoftwo}{\hi@ifset#2}%
}

```

4 Citation Groups

\defcitegroup Often there will be several references that follow a standard conventional form, so it would be convenient to define all of them in one go. The `\defcitegroup` macro enables this to an extent. It takes four arguments:

- (optional) A prefix for the reference nicknames to be defined, default blank
- A reference definer macro (`\def<type>`)
- Initial text for the reference definition body
- A comma-separated list of texts used both as the nickname and as a suffix to the reference definition

For example, say you want to define references for several sections of the Patent Act. That could be achieved with the following three definitions:

```
\defstatcode{101}{35 U.S.C. S 101}
\defstatcode{102}{35 U.S.C. S 102}
\defstatcode{103}{35 U.S.C. S 103}
```

This works (using the reference parser for statutes) but is repetitive. Instead, you can write:

```
\defcitegroup\defstatcode{35 U.S.C. S }{101, 102, 103}
```

Each of the items in the comma-separated list (e.g., 101) becomes both a reference nickname and the end of the reference definition.

With the optional argument, the reference names can be given a prefix:

```
\defcitegroup[17usc]\defstatcode{17 U.S.C. S }{101,
102, 103}
```

would define `17usc101` as the statute “17 U.S.C. § 101” and so on.

```
\newcommand\defcitegroup[4][]{%
  \forcsvlist{\hi@citegroup@{#1}{#2}{#3}}{#4}%
}
\def\hi@citegroup@#1#2#3#4{#2{#1#4}{#3#4}}
```